

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA1418

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:50

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Y. Abhay-192411219 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



ANALYTICAL PROBLEM SOLVING

1. Trace the compilation process

Consider the following C statement:

```
total = price + quantity * 10;
```

Trace this statement through the relevant phases of a compiler by identifying the lexemes and tokens produced during lexical analysis, explaining how the syntax analyzer uses the token stream, describing the role of semantic analysis, and showing how an appropriate intermediate representation could be generated. Finally, explain how the grouping of compiler phases into front end, middle end, and back end helps in processing this statement.

2. Analyze token recognition and ambiguity

Consider the following input statement:

```
int count = a+++b;
```

Analyze how a lexical analyzer should divide the input into lexemes and tokens. Identify the possible ambiguity in recognizing the sequence `+++`, explain how longest-match/token recognition rules influence the result, and propose suitable token specifications for identifiers, keywords, assignment operators, increment operators, and arithmetic operators. Justify why your proposed tokenization is appropriate for the given language.

3. Determine and classify tokens

Consider the following program:

```
/* Calculate the average */
int main()
{
    int sum = 100, count = 5;
    float average = sum / count;
    return 0;
}
```

Determine the total number of tokens generated by the lexical analyzer. Analyze the program line by line and classify each token as a keyword, identifier, constant/literal, operator, separator, or punctuation symbol. Explain how comments and whitespace are handled by the lexical analyzer and justify the final token count.

4. Design token specifications using regular expressions

A simplified programming language contains the following token categories:

- Keywords: `if, else, while, return`
- Identifiers: begin with a letter and may be followed by letters or digits
- Integer constants: one or more digits
- Relational operators: `<, <=, >, >=, ==, !=`
- Arithmetic operators: `+, -, *, /`
- Separators: `(, ), {, }, ;`

For the input:

```
if (count >= 10) {
    total = count + 25;
```

```
    return total;
}
```

design appropriate regular-expression specifications for the token categories, trace the input through the lexical analyzer, and produce the complete token stream. Explain how the lexical analyzer distinguishes keywords from identifiers and how overlapping operators such as < and <= are recognized correctly.

5. Analyze and implement a LEX specification

A college wants to develop a lexical analyzer using LEX for a simple language containing:

- Keywords: int, float, if, else, return
- Identifiers
- Integer constants
- Floating-point constants
- Arithmetic operators: +, -, *, /
- Relational operators: <, <=, >, >=, ==
- Assignment operator: =
- Separators: (,), {, }, ;

For the input:

```
float average = total / 5.0;
if (average >= 50) return average;
```

Analyze the tokenization requirements and develop a LEX specification containing suitable regular-expression patterns and actions for recognizing the above token categories. Explain the purpose of the declaration section, rule section, and auxiliary procedure section of the LEX program. Finally, describe the sequence of commands required to generate lex.yy.c, compile it, and execute the resulting lexical analyzer.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear or interpretation	No interpretation

Answers

1. Trace the Compilation Process

Given Statement

total = price + quantity * 10;

The compilation of this statement passes through lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and code generation.

1. Lexical Analysis

The lexical analyzer reads the source program character by character and groups characters into **lexemes**. Each lexeme is assigned a **token**.

Lexeme	Token
total	Identifier
=	Assignment Operator
price	Identifier
+	Arithmetic Operator
quantity	Identifier
*	Arithmetic Operator
10	Integer Constant
;	Separator

Therefore, the token stream can be represented as:

<ID,total> <ASSIGN> <ID,price> <PLUS> <ID,quantity> <MUL> <NUM,10> <SEMICOLON>

2. Syntax Analysis

The syntax analyzer, or **parser**, receives the token stream from the lexical analyzer and checks whether the tokens follow the grammar of the programming language.

The statement follows the structure:

assignment	→	identifier	=	expression	;
expression	→	expression	+	term	
term	→	term	*	factor	
factor	→ identifier constant				

The parser recognizes that:

total = price + quantity * 10;

is a valid assignment statement.

Importantly, multiplication has higher precedence than addition, so the expression is interpreted as:

total = price + (quantity * 10);

A simplified syntax tree is:



3. Semantic Analysis

Semantic analysis checks whether the statement is **meaningful according to the language rules**.

For example, it verifies:

- total, price, and quantity have been declared.
- Their types are compatible.
- The multiplication operation between quantity and 10 is valid.
- The addition of price and quantity * 10 is valid.
- The resulting value can be assigned to total.

For example, if:

int total, price, quantity;

then the expression is semantically valid.

4. Intermediate Code Generation

The compiler can convert the expression into an intermediate representation such as **Three-Address Code (TAC)**:

```
t1          =          quantity          *          10
t2          =          price          +          t1
total = t2
```

Here, t1 and t2 are temporary variables.

5. Grouping Compiler Phases

The phases can be grouped into three major parts:

Compiler Part	Relevant Work
Front End	Lexical analysis, syntax analysis, semantic analysis
Middle End	Intermediate representation and optimization
Back End	Target code generation

For this statement:



This grouping makes compiler construction more systematic. The **front end** understands the source program, the **middle end** works on an intermediate representation, and the **back end** produces code for the target machine.

2. Analyze Token Recognition and Ambiguity

Given Input

int count = a+++b;

The lexical analyzer must divide the input into meaningful lexemes and tokens.

Tokenization

A suitable tokenization is:

int	→	KEYWORD
count	→	IDENTIFIER
=	→	ASSIGNMENT
a	→	IDENTIFIER
++	→	INCREMENT
+	→	PLUS
b	→	IDENTIFIER
;	→	SEMICOLON

Therefore:

<KEYWORD,int>
<ID,count>
<ASSIGN>
<ID,a>
<INCREMENT>
<PLUS>
<ID,b>
<SEMICOLON>

Ambiguity in +++

The sequence:

+++

can potentially be viewed as:

++ +

or:

+ ++

The lexical analyzer needs a rule to determine which interpretation is selected.

Longest-Match Rule

Most lexical analyzers use the **longest-match rule**, also called the **maximal munch rule**.

This means that when multiple patterns can match the input, the lexer selects the longest valid token.

Since:

`++`

is a valid increment operator and is longer than:

`+`

the sequence is divided as:

`++ +`

Thus:

`a+++b`

becomes:

`a ++ + b`

However, the exact interpretation can depend on the lexical and syntactic rules of the particular programming language. The token specification must therefore clearly define how `++` and `+` interact.

Suitable Token Specifications

Identifier:

`[A-Za-z][A-Za-z0-9]*`

Keyword:

`int`

Assignment operator:

`=`

Increment operator:

`\+\+`

Arithmetic plus operator:

`\+`

Semicolon:

;

The keyword rule should be recognized before the general identifier rule, or the lexer should classify an identifier such as `int` as a keyword after matching it.

Justification

The proposed tokenization is appropriate because:

1. `int` is a reserved keyword.
2. `count` and `a` are identifiers.
3. `=` represents assignment.
4. `++` is recognized as the increment operator.
5. The remaining `+` is recognized as the arithmetic addition operator.
6. `b` is an identifier.
7. `;` terminates the statement.
8. The longest-match rule prevents the lexer from incorrectly splitting `++` into two separate `+` tokens.

3. Determine and Classify Tokens

Given Program

```
/*          Calculate          the          average          */  
  
int main()  
{  
    int sum = 100, count = 5;  
    float average = sum / count;  
    return 0;  
}
```

The comment is not converted into a token because comments are ignored by the lexical analyzer.

Line-by-Line Analysis

Line 1

```
/* Calculate the average */
```

This is a comment.

Tokens generated: 0

Line 2

```
int main()
```

Lexeme	Classification
int	Keyword
main	Identifier
(	Separator/Punctuation
)	Separator/Punctuation

Tokens: 4

Line 3

```
{
```

Lexeme	Classification
{	Separator/Punctuation

Tokens: 1

Line 4

```
int sum = 100, count = 5;
```

Lexeme	Classification
int	Keyword
sum	Identifier
=	Operator
100	Constant/Literal
,	Separator
count	Identifier
=	Operator
5	Constant/Literal
;	Separator

Tokens: 9

Line 5

```
float average = sum / count;
```

Lexeme	Classification
float	Keyword
average	Identifier
=	Operator
sum	Identifier
/	Operator
count	Identifier
;	Separator

Tokens: 7

Line 6

```
return 0;
```

Lexeme	Classification
return	Keyword
0	Constant/Literal
;	Separator

Tokens: 3

Line 7

```
}
```

Lexeme	Classification
}	Separator/Punctuation

Tokens: 1

Total Token Count

Line	1	=	0
Line	2	=	4
Line	3	=	1
Line	4	=	9
Line	5	=	7
Line	6	=	3
Line	7	=	1

Total	= 25?		

Let's carefully count the actual token stream:

1.	int
2.	main
3.	(
4.	)
5.	{
6.	int
7.	sum
8.	=
9.	100
10.	,
11.	count
12.	=
13.	5
14.	;
15.	float
16.	average
17.	=
18.	sum
19.	/
20.	count
21.	;
22.	return
23.	0
24.	;
25. }	

Therefore, the **total number of tokens is 25**.

Handling Comments and Whitespace

Comments:

The comment:

```
/* Calculate the average */
```

is removed/ignored during lexical analysis and contributes **zero tokens**.

Whitespace:

Spaces, tabs, and newlines generally separate lexemes but are not themselves returned as tokens. For example:

```
int sum
```

```
and
```

```
int  sum
```

produce the same token sequence.

4. Design Token Specifications Using Regular Expressions

Token Categories

The simplified language contains the following categories.

1. Keywords

if|else|while|return

2. Identifiers

An identifier begins with a letter and may be followed by letters or digits:

[A-Za-z][A-Za-z0-9]*

3. Integer Constants

[0-9]+

4. Relational Operators

<=|>|=|!=|<|>

The longer operators should be placed before their shorter versions.

5. Arithmetic Operators

\+|-|*|/

6. Separators

[\(\)\{\}\;]

Given Input

if	(count	>=	10)	{
total	=	count	+	25;
return				total;
}				

Complete Token Stream

<IF>
<LPAREN>
<ID,count>
<GE>
<NUM,10>
<RPAREN>
<LBRACE>

<ID,total>
<ASSIGN>
<ID,count>
<PLUS>
<NUM,25>
<SEMICOLON>

<RETURN>
<ID,total>
<SEMICOLON>

<RBRACE>

Token-by-Token Table

Lexeme	Token
if	Keyword
(	Separator
count	Identifier
>=	Relational Operator
10	Integer Constant
)	Separator
{	Separator
total	Identifier
=	Assignment Operator
count	Identifier
+	Arithmetic Operator
25	Integer Constant
;	Separator
return	Keyword
total	Identifier
;	Separator
}	Separator

Keyword vs Identifier

The lexical analyzer first matches a sequence such as:

if

against the identifier pattern. Since if is present in the reserved keyword table, it is classified as a **keyword**.

Similarly:

count
total

are not reserved words, so they are classified as identifiers.

Handling < and <=

Consider:

<
<=

Both are possible matches. The lexer uses the **longest-match rule**.

Therefore, when the input contains:

<=

the lexer recognizes the complete two-character relational operator rather than recognizing < followed by =.

Thus, operator rules should preferably specify:

<=
<

rather than placing < before <=.

5. Analyze and Implement a LEX Specification

Given Language

The language contains:

- Keywords: int, float, if, else, return
- Identifiers
- Integer constants
- Floating-point constants
- Arithmetic operators
- Relational operators
- Assignment operator
- Separators

Given Input

```
float          average          =          total          /          5.0;

if (average >= 50) return average;
```

The expected tokenization is:

FLOAT		→	float
ID		→	average
ASSIGN		→	=
ID		→	total
DIV		→	/
FLOAT_CONST	→		5.0
SEMICOLON		→	;
IF		→	if
LPAREN		→	(
ID		→	average
GE		→	>=
INT_CONST	→		50
RPAREN		→	)
RETURN		→	return
ID		→	average
SEMICOLON	→		;

LEX Specification

```
%{

#include <stdio.h>

%}
```

```

%%

"int" { printf("KEYWORD: int\n"); }

"float" { printf("KEYWORD: float\n"); }

"if" { printf("KEYWORD: if\n"); }

"else" { printf("KEYWORD: else\n"); }

"return" { printf("KEYWORD: return\n"); }

[0-9]+.[0-9]+ { printf("FLOAT_CONST: %s\n", yytext); }

[0-9]+ { printf("INT_CONST: %s\n", yytext); }

">=" { printf("REL_OP: >=\n"); }

"<=" { printf("REL_OP: <=\n"); }

"==" { printf("REL_OP: ==\n"); }

">" { printf("REL_OP: >\n"); }

"<" { printf("REL_OP: <\n"); }

"+" { printf("ARITH_OP: +\n"); }

"-" { printf("ARITH_OP: -\n"); }

"*" { printf("ARITH_OP: *\n"); }

"/" { printf("ARITH_OP: /\n"); }

"=" { printf("ASSIGN: =\n"); }

"(" { printf("SEPARATOR: (\n"); }

")" { printf("SEPARATOR: )\n"); }

"{" { printf("SEPARATOR: {\n"); }

"}" { printf("SEPARATOR: }\n"); }

";" { printf("SEPARATOR: ;\n"); }

[A-Za-z][A-Za-z0-9]* { printf("IDENTIFIER: %s\n", yytext); }

[ \t\n]+ ; . { printf("UNKNOWN: %s\n", yytext); }

%%

int main() { yylex(); return 0; }

```

Important Note

The keyword rules are placed **before the general identifier rule**. Therefore, when the lexer encounters:

float
if
return

they are recognized as keywords rather than identifiers.

The relational operators such as:

>=
<=
==

are also placed before the single-character operators so that the longer operators are recognized correctly.

Structure of a LEX Program

A LEX program has three major sections:

Definitions

%%

Rules

%%

Auxiliary Procedures

1. Declaration/Definition Section

This section appears before the first %%.

Example:

```
%{  
#include <stdio.h>  
%}
```

It contains C declarations, header files, variables, and other definitions required by the generated scanner.

2. Rule Section

The rule section contains:

pattern action

For example:

```
[0-9]+ { printf("INTEGER: %s\n", yytext); }
```

The regular expression identifies the input pattern and the action specifies what the lexical analyzer should do when the pattern is matched.

3. Auxiliary Procedure Section

This section appears after the second %%.

For example:

```
int                                     main()
{
    yylex();
    return                             0;
}
```

It contains supporting C functions such as main() and other user-defined functions.

Commands to Generate and Execute the LEX Analyzer

Suppose the LEX program is saved as:

scanner.l

Step 1: Generate lex.yy.c

lex scanner.l

This generates:

lex.yy.c

Step 2: Compile the generated C program

Using GCC:

gcc lex.yy.c -o scanner

Step 3: Execute

./scanner

On Windows, depending on the LEX/FLEX environment, the executable can be run as:

scanner.exe

Then provide the input:

```
float          average          =          total          /          5.0;
if (average >= 50) return average;
```

The lexical analyzer identifies each lexeme and prints its corresponding token category.